

Papers Dock



PAPER 4

PRACTICE PAPER 3

Open the document **evidence.doc**.

Make sure that your name, centre number and candidate number will appear on every page of this document. This document will contain your answers to each question.

Save this evidence document in your work area as:

evidence_ followed by your centre number_candidate number, for example:
evidence_zz999_9999

A class declaration can be used to declare a record.

A list is an alternative to an array.

1) A program needs to use a stack data structure. The stack can store up to 10 integer elements

A 1D array StackData is used to store the stack globally. The global variable StackPointer points to the next available space in the stack and is initialised to 0.

(a) Write program code to declare the array and pointer as global data structures. Initialise the pointer to 0

Save your program as **question1**.

Copy and paste the program code into part 1(a) in the evidence document.

[3]

(b) Write a procedure to output all 10 elements in the stack and the value of StackPointer.

Save your program.

Copy and paste the program code into part 1(b) in the evidence document.

[3]

(c) The function Push() takes an integer parameter and returns FALSE if the stack is full. If the stack is not full, it puts the parameter value on the stack, updates the relevant pointer and returns TRUE.

Write program code for the function Push().

Save your program.

Copy and paste the program code into part 1(c) in the evidence document.

[6]

(d) (i) Edit the main program to test the Push() function. The main program needs to:

- allow the user to enter 11 numbers and attempt to add these to the stack
- output an appropriate message when a number is added to the stack
- output an appropriate message when a number is not added to the stack if it is full
- output the contents of the stack after attempting to add all 11 numbers.

Save your program.

Copy and paste the program code into part 1(d)(i) in the evidence document.

[5]

(ii) Test your program from part 1(d)(i) with the following 11 inputs:

11 12 13 14 15 16 17 18 19 20 21

Take a screenshot to show the output.

Copy and paste the screenshots into part **1(d)(ii)** in the evidence document.

[1]

(e) The function Pop() returns -1 if the stack is empty. If the stack is not empty, it returns the element at the top of the stack and updates the relevant pointer.

(i) Write program code for the function Pop().

Save your program.

Copy and paste the program code into part **1(e)(i)** in the evidence document.

[5]

(ii) After the code you wrote in the main program for part 1(d)(i), add program code to:

- remove two elements from the stack using Pop()
- output the updated contents of the stack.

Test your program and take a screenshot to show the output.

Copy and paste the screenshots into part **1(e)(ii)** in the evidence document.

[2]

- 2 Ejaz is creating a program that will allow the user to create quizzes. He is using object-oriented programming (OOP).

There are two classes: `QuestionClass` and `QuizClass`.

The class attributes and methods are in the following tables. All attributes are declared as private.

QuestionClass	
<code>Question : STRING</code>	<code>// stores the question</code>
<code>Answer : STRING</code>	<code>// stores the correct answer</code>
<code>Difficulty : INTEGER</code>	<code>// stores the difficulty as an integer</code> <code>// from 0(easy) to 10(hard)</code>
<code>Constructor(QuestionP, AnswerP, DifficultyP)</code>	<code>// creates an instance of QuestionClass</code> <code>// sets the attributes to the parameter</code> <code>// values</code>
<code>GetQuestion()</code>	<code>// returns the question</code>
<code>GetDifficulty()</code>	<code>// returns the difficulty level</code>
<code>GetAnswer()</code>	<code>// returns the answer</code>

QuizClass	
<code>Questions : ARRAY[0:19] OF QuestionClass</code>	<code>// stores maximum 20 questions of</code> <code>// type QuestionClass</code>
<code>NumberOfQuestions : INTEGER</code>	<code>// stores the number of questions</code> <code>// in this quiz</code>
<code>Constructor()</code>	<code>// creates an instance of</code> <code>// QuizClass</code> <code>// initialises NumberOfQuestions</code> <code>// to 0</code>
<code>AddQuestion()</code>	<code>// adds the parameter question to</code> <code>// the array</code> <code>// increments NumberOfQuestions</code>
<code>GetQuestion()</code>	<code>// returns the next question to be</code> <code>// asked</code>
<code>CheckAnswer()</code>	<code>// takes an answer as a parameter</code> <code>// and returns TRUE if correct</code>

(a) Write Program Code to define the class `QuestionClass`. You are required to write the code for the attribute declarations, constructor and the getters.

If you are writing in Python, include attribute declarations using comments.

Use your programming language's constructor method.

Save your program in a new file with name **Question2**

Copy and paste the program code into part **2(a)** in the evidence document.

[6]

(b) Write **program code** to define the class `QuizClass`. You are only required to write code for the attribute declarations and constructor.

If you are writing in Python, include attribute declarations using comments.

Use your programming language's constructor method.

Save your program.

Copy and paste the program code into part **2(b)** in the evidence document.

[4]

(c) The `QuizClass` method `AddQuestion()` takes a question object as a parameter and stores it in the next available location in the array `Questions`. It returns `TRUE` if it is successfully stored, and `FALSE` otherwise.

Write **program code** for the method `AddQuestion()`.

Save your program.

Copy and paste the program code into part **2(c)** in the evidence document.

[4]

(d) The first quiz is created with the identifier `FirstQuiz`.

The first question in this quiz is: "What is 100 / 5 ?".

The answer is "20" and the difficulty level is 1.

Write **program code** to:

- declare an instance of `QuizClass` with the identifier `FirstQuiz`
- declare an instance of `QuestionClass` with the identifier `Question1`
- add `Question1` to the array in `FirstQuiz` using `AddQuestion()`.

Save your program.

Copy and paste the program code into part **2(d)** in the evidence document.

[5]

(3) 1 A program stores the following 10 numbers in a 1D array with identifier numData.

10 4 23 45 12 8 21 11 3 1

(a) Write program code for a **new program** to:

- declare the global 1D array, numData, with 10 elements
- initialise numData in the main program using the data values shown

Save your program as **question3**.

Copy and paste the program code into part 1(a) in the evidence document.

[2]

(b) (i) A function BubbleSort() takes numData as by reference parameter and sort the elements in the numData array in ascending order.

Write program code for the function BubbleSort() and make sure the code is efficient.

Save your program.

Copy and paste the program code into part 3(b)(i) in the evidence document.

(ii) Test your program by calling the function BubbleSort().

Take a screenshot to show the result

Save your program.

Copy and paste the screenshots into part 3(b)(ii) in the evidence document.

[1]

(c) The Function EvenNumber() takes the array as a Parameter and it returns count of how numbers are even.

(i) Write program code for the function EvenNumber().

Save your program.

Copy and paste the program code into part 3(c)(i) in the evidence document.

[3]

(ii) Pass the numData in the Function EvenNumber() and find out how many even numbers are there in numData and print it with suitable message.

Save your program.

Copy and paste the program code into part 3(c)(ii) in the evidence document.

[3]

(iii) Take the screenshot of the output of **(ii)**

Save your program.

Copy and paste the screenshots into part 3(c)(iii) in the evidence document.

[1]